A Polynomial Time Algorithm for Finding a Minimum 4-Partition of a Submodular Function*

Tsuyoshi Hirayama† Yuhao Liu‡ Kazuhisa Makino§ Ke Shi¶ Chao Xu‖**

Abstract

In this paper, we study the minimum k-partition problem of submodular functions, i.e., given a finite set V and a submodular function $f : 2^V \to \mathbb{R}$, computing a k-partition $\{V_1, \ldots, V_k\}$ of V with minimum $\sum_{i=1}^{k} f(V_i)$. The problem is a natural generalization of the minimum k-cut problem in graphs and hypergraphs. It is known that the problem is NP-hard for general k, and solvable in polynomial time for $k \leq 3$. In this paper, we construct the first polynomial-time algorithm for the minimum 4-partition problem.

1 Introduction

Let V be a finite set with $n = |V|$, and let $f : 2^V \to \mathbb{R}$ be a *submodular* function, i.e., $f(X) + f(Y) \geq f(X \cap Y) + f(X \cup Y)$ holds for any $X, Y \subseteq V$. For an integer $k \geq 2$, the minimum k-partition problem for a submodular function f is to compute a k-partition $\mathcal{P}_k = \{V_1, \ldots, V_k\}$ of V with the minimum value defined as $\sum_{i=1}^{k} f(V_i)$, where, for a positive integer k, $\{V_1, \ldots, V_k\}$ is called a *k-partition* if $V_i \neq \emptyset$ for all i, $\bigcup_{i=1}^{k} V_i = V$, and $V_i \cap V_j = \emptyset$ for all i and j with $i \neq j$. This is one of the most fundamental problems in combinatorial optimization, and a natural generalization of the minimum k-cut problem in graphs and hypergraphs, where both problems are polynomial time solvable for fixed k.

The minimum k-cut problem has many applications such as the traveling salesperson problem, VLSI design, evolutionary tree construction and network reliability [12, 22]. Goldschmidt-Hochbaum [13] showed that the minimum k-cut problem in graphs is NP-hard, when k is a part of input, but it can be solved in polynomial time for fixed k. After their work, a number of algorithms were proposed; See [8, 16, 17, 22, 23, 24, 28, 30], for example. The current best deterministic algorithm has $\tilde{O}(mn^{k-1})$ time for $k \leq 6$ [21, 23, 24, 32], $\tilde{O}(mn^{2k-2})$ time for $k \geq 7$ [8], $k^{O(k)} n^{(2\omega/3+\epsilon)k+O(1)}$ for any $\epsilon > 0$ and polynomially bounded weights, where ω is the matrix multiplication constant [16], and a randomized algorithm in $\tilde{O}(n^k)$ time [15].

The minimum k-cut problem in hypergraphs is NP-hard, which immediately follows from the NP-hardness of the graph problem [13]. Klimmek and Wagner [18] and Mak and Wong [20] showed that the minimum 2-cut problem in hypergraphs can be solved in $\tilde{O}(dn)$ time, where d denotes the sum of the cardinalities of all hyperedges. Chekuri and Xu showed that in the same running time, they can count and enumerate all minimum 2-cuts [9]. For $k = 3$, Xiao [31] constructed an $\tilde{O}(d(m+n)n^3)$-time algorithm. Fukunaga [11] showed that the problem can be solved in polynomial time if both k and $\max_{e \in E} |e|$ are fixed. A randomized polynomial time algorithm was found by Chandrasekaran et. al. [6].

*Authors are ordered alphabetically.

†tsuyoshi1.hirayama@toshiba.co.jp, Toshiba Digital Solutions Corporation.
‡yuhaoliu126@gmail.com, University of Electronic Science and Technology of China.
§makino@kurims.kyoto-u.ac.jp, Kyoto University.
¶self.ke.shi@gmail.com, University of Electronic Science and Technology of China.
‖the.chao.xu@gmail.com, University of Electronic Science and Technology of China.
**Corresponding author

Later works speed up the randomized algorithm [10], and generalize to multicriteria objective and size constraints [3]. Recently, Chandrasekaran and Chekuri finally settled the open problem and showed minimum k-cut problem in hypergraphs is polynomial time solvable for each fixed k [5]. There is some follow-up work on counting and enumerating all minimum k-cuts [1, 2].

The minimum k-partition problem for fixed k is much less understood. A submodular function is symmetric, if $f(X) = f(V \setminus X)$. For $k = 2$, the problem is equivalent to the symmetric submodular function minimization, since $f(X)$ can be replaced with $\frac{1}{2}(f(X) + f(V - X))$. Hence it can be solved in $O(n^3\gamma)$ time by Queyranne's algorithm [27], where γ denotes the time required for function evaluation. For $k = 3$, Okumoto et al. [26] presented an $O(n^3\tau(n))$-time algorithm by extending Xiao's algorithm for the minimum 3-cut problem for hypergraphs [31], where $\tau(n)$ denotes the time required to solve the submodular function minimization, and the current best known bound for $\tau(n)$ is $\tilde{O}(n^3\gamma + n^4)$ [19]. However, it is still *open* if there exists a polynomial time algorithm for fixed $k \geq 4$ [26]. It was implied in [4] that symmetric submodular k-partition reduces to n^{2k-2} calls of submodular $(k-1)$-partition. There are several studies on approximation algorithms for the problem [7, 26, 33, 34]. The current best approximation ratio is 1.5 for $k = 4$ [26] and 2 for $k \geq 5$ [7].

Our results We show the following two results.

1. An $O(n^6\tau(n))$ time algorithm for finding a minimum 4-partition of a submodular function.
2. As a corollary, an $O(n^{14}\tau(n))$ time algorithm for finding minimum 5-partition of a *symmetric* submodular function.

This settles the complexity status of the submodular k-partition problem for $k = 4$ [11, 22, 23, 26, 31, 34]. Our algorithm is based on the *compatibility* of 3- and 4-partitions, which is a generalization of the noncrossing property of 2- and 3-partitions proposed in [14, 26, 31]. There exist two natural and possible extensions of their noncrossing property. However, both extensions fail to produce a polynomial time solution to the minimum 4-partition problem (see the detailed discussion in Section 3).

The rest of the paper is organized as follows. In Section 3, we present the compatibility of 3- and 4-partitions, where the proof can be found in Section 4, and describe our algorithm. In Section 5, we present how to compute a minimum nontrivial 3-partition, which is a subroutine of our algorithm. Section 6 analyzes the time complexity of our algorithm. Finally in Section 7, we conclude with some remarks.

2 Preliminaries

We write $\binom{V}{i}$ to be the family of all size i subsets of V. We abuse the notation to generalize each function f on sets to a function on partitions. That is, for any partition $\mathcal{P}$, we define $f(\mathcal{P}) = \sum_{X \in P} f(X)$. Let $g : 2^V \to \mathbb{R}$ be a set function. For a set $U \subseteq V$, let $g_{\setminus U}$ denote a set function obtained from g by deleting U from V, and $g_{/U}$ denote a set function obtained from g by shrinking U into a new element u (i.e., $u \notin V$). Namely, $g_{\setminus U} : 2^{V \setminus U} \to \mathbb{R}$ satisfies $g_{\setminus U}(S) = g(S)$, and $g_{/U} : 2^{(V \setminus U) \cup \{u\}} \to \mathbb{R}$ satisfies $g_{/U}(S) = g(S)$ if $S \not\ni u$, and $g((S \setminus \{u\}) \cup U)$, otherwise. We note that $g_{\setminus U}$ and $g_{/U}$ are both submodular if g is submodular.

For two partitions $\mathcal{X} = \{X_1, \ldots, X_k\}$ and $\mathcal{Y} = \{Y_1, \ldots, Y_m\}$, a matrix M is a *configuration* of $\mathcal{X}$ and $\mathcal{Y}$, if $M_{i,j} = |X_i \cap Y_j|$. We will abuse the notation and write M_{ij} when it will not lead to confusion. We say configuration M is *generated* by $\mathcal{X}$ and $\mathcal{Y}$.

Two configurations M_1 and M_2 are equivalent if there exist a row permutation and a column permutation to swap M_1 into M_2. Configurations help us visualize the ways partitions intersect.

To represent a set of possible configurations M visually, we write a few numbers in the matrix to indicate what pattern the configuration matches. We use the number i to denote the values that are known to be i, i^+ if the value known to be at least i, i^- if the value known to be at most i, and ? means either we don't know or we don't care about its value.

3 Compatibility for 3- and 4-partitions

In this section, we present the compatibility of minimum 3- and 4-partitions in submodular functions, from which we derive a polynomial time algorithm for the minimum 4-partition problem. We start with the noncrossing property for 2- and 3-partitions, which was proven in [26].

A set X is *noncrossing* with a partition $\mathcal{Y}$, if $X \subseteq Y$ for some $Y \in \mathcal{Y}$. A partition $\mathcal{X}$ *is noncrossing with* a partition $\mathcal{Y}$, if there exists a component $X \in \mathcal{X}$ that is noncrossing with $\mathcal{Y}$. A partition is called *h-size* if all its components contain at least h elements. A partition $\mathcal{X}$ is called *trivial* if all but one component have exactly 1 element. For 2-partitions over a ground set with at least 4 elements, it is trivial if and only if it is not 2-size.

The following lemma is a direct consequence of Corollary 1 from [26].

Lemma 3.1 *Let $f : 2^V \to \mathbb{R}$ be a submodular function with $n \geq 7$, and let $\mathcal{X}$ denote a minimum non-trivial 2-partition of f. If f has a minimum 3-partition of 2-size, then it contains a minimum 3-partition which is noncrossing with $\mathcal{X}$.*

By this lemma, we can easily construct the following divide-and-conquer algorithm for the minimum 3-partition problem [26], see Figure 3.1.

```
MIN3PARTITION(f):
  V ← domain(f)
  if |V| ≤ 6
      return the optimum by brute force
  for X ∈ (V choose 1)
      add candidate {X} ∪ MIN2PARTITION(f_{\X})
  𝒳 ← MINNONTRIVIAL2PARTITION(f)
  for X ∈ 𝒳
      add candidate MIN3PARTITION(f_{/X})
  return minimum over all candidates
```

Figure 3.1: Compute a minimum 3-partition.

We first compute some candidate partitions, where one of them is a minimum non-2-size 3-partition of f (i.e., a minimum partition of type $\{\{v\}, W_1, W_2\}$ for some $v \in V$). To do this, we compute a minimum partition $\{W_1, W_2\}$ of $f_{\setminus\{v\}}$ for each $v \in V$. We next compute a minimum 2-size 2-partition $\mathcal{X} = \{X_1, X_2\}$ and recursively call the algorithm for functions $f_{/X_1}$ and $f_{/X_2}$ and obtain two candidate partitions. Finally, we take the minimum of all candidate partitions. Therefore, the noncrossing property in Lemma 3.1 produces a polynomial time algorithm for the minimum 3-partition problem.

Let us consider possible generalizations of the noncrossing property. The first one is a stronger property for 2- and 4-partitions. Let $\mathcal{X} = \{X_1, X_2\}$ denote a minimum 2-partition of f. There exists a minimum 4-partition $\mathcal{Y} = \{Y_1, Y_2, Y_3, Y_4\}$ of f such that either $\mathcal{X}$ is noncrossing with $\mathcal{Y}$ or $X_1 = Y_1 \cup Y_2$ after some renumbering of the indices. We remark that $\mathcal{X}$ is *not* necessarily h-size for $h \geq 2$, and hence the property above does not provide a polynomial divide-and-conquer algorithm for the minimum 4-partition problem. If $\mathcal{X}$ is assumed to be h-size for $h \geq 2$, then we have additional cases, one of which also blocks to construct a polynomial time algorithm (See the case (iii) in Theorem 6 in [25]).

The second generalization is to directly use noncrossing property for 3- and 4-partitions. Assume that every minimum h-size 3-partition $\mathcal{X} = \{X_1, X_2, X_3\}$ for $h \geq 2$ is noncrossing with a minimum 4-partition. However, this does not provide a polynomial divide-and-conquer algorithm for the minimum 4-partition problem. Note that the algorithm calls itself for $f_{/X_i}$, $i = 1, 2, 3$, where in the worst case, two of the recursive calls have size $n-1$, this implies that a simple divide-and-conquer algorithm requires exponential time.

In this paper, we introduce the concept of compatibility to overcome this difficulty. A partition $\mathcal{X}$ is *compatible* with partition $\mathcal{Y}$, if $|\mathcal{X}|-1$ components of $\mathcal{X}$ is noncrossing with $\mathcal{Y}$. We write $\mathcal{X} \lhd \mathcal{Y}$. Note if $\mathcal{X}$ and $\mathcal{Y}$ are 2- and 3- partitions, respectively, then the compatibility relation is identical to the noncrossing property.

Theorem 3.2 *Let f be a submodular function on at least 13 vertices. If all minimum 4-partitions are 3-size, then every minimum non-trivial 3-partition is compatible with some minimum 4-partition.*

Compatibility is a very strong property about two partitions, and it is very unlikely to hold true in general. In fact, there are examples where a minimum 4-partition is not compatible with any minimum 5-partition, as we will see in section 7.

The proof of Theorem 3.2 can be found in the next section. We remark that the proof is based on case analysis. Based on Theorem 3.2, it is not difficult to see that the following simple contraction based algorithm solves the minimum 4-partition problem. We invite the readers to spot the difference between the minimum 3-partition algorithm in Figure 3.1 and the minimum 4-partition algorithm in Figure 3.2.

3.1 The algorithm

Either there is a minimum 4-partition that is not 3-size, so there is a part of size at most 2. We try all possible such parts, and solve the minimum 3-partition problem on the remaining part. Otherwise, there is a 3-size minimum 4-partition. We find a minimum non-trivial 3-partition, and we know it is compatible to some minimum 4-partition by Theorem 3.2. Therefore we can contract two of the parts. Since we do not know which, we try all possibilities. Finally, we collect all candidates we computed, and find the minimum among them. The full algorithm is described in Figure 3.2.

```
MIN4PARTITION(f):
  V ← domain(f)
  if |V| ≤ 12
      return the optimum by brute force
  for X ∈ (V choose 1) ⋃ (V choose 2)
      add candidate {X} ∪ MIN3PARTITION(f_{\X})
  𝒳 ← MINNONTRIVIAL3PARTITION(f)
  for {A,B} ∈ (𝒳 choose 2)
      add candidate MIN4PARTITION((f_{/A})_{/B})
  return minimum over all candidates
```

Figure 3.2: Compute a minimum 4-partition.

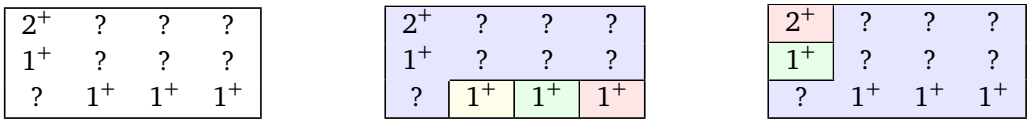
We computed $O(n^2)$ minimum 3-partitions, and computed a minimum non-trivial 3-partition. The remaining non-recursive operations take constant time per statement.

The minimum 3-partition problem is solvable in polynomial time as we noted in this section. Thus, except for the proof of Theorem 3.2, what remains to be done is to provide a polynomial time algorithm to compute a minimum *nontrivial* 3-partition, which is discussed in Section 5.

4 Proof of Theorem 3.2

Let V be the ground set of some submodular function f. Consider any 3-partition $\mathcal{X} = \{X_1, X_2, X_3\}$ and $\mathcal{Y} = \{Y_1, Y_2, Y_3, Y_4\}$. A submodular function f on at least 13 vertices where every minimum 4-partition is 3-size, is called a *valid* submodular function. A configuration is *valid*, if it can be generated by a minimum non-trivial 3-partition $\mathcal{X}$, and a minimum 4-partition $\mathcal{Y}$ of some valid submodular function f.

Let M be some configuration. Consider a submodular function f, such that it has a minimum non-trivial 3-partition $\mathcal{X}$ and a 3-size minimum 4-partition $\mathcal{Y}$ that generate M. Such function are called



(a) M (b) $\mathcal{Y}'$ (c) $\mathcal{X}'$

Figure 4.1: The configuration M and partitions $\mathcal{Y}'$ and $\mathcal{X}'$ in the proof of Lemma 4.2. To understand the illustration of $\mathcal{Y}'$, note it is an colored overlay over the configuration matrix M. Each color represents a partition class of $\mathcal{Y}'$. Since the ith row and jth column represents $Z_{ij} = X_i \cap Y_j$ together with additional cardinally information, the illustration shows $\mathcal{Y}'$ is a 4-partition with partition classes Z_{32}, Z_{33}, Z_{34} and a set containing all the remaining elements.

M-agreeable. If there is a minimum 4-partition $\mathcal{Y}'$ of f that $\mathcal{X}$ is compatible with, then we say M is f-good. If M is f-good for all M-agreeable f, we say M is *good*. If M is good, then any configuration equivalent to M is also good.

If all valid configurations are good, then Theorem 3.2 is true. Indeed, for any valid submodular function, the minimum non-trivial 3-partition and 3-size minimum 4-partition generates a good configuration, which implies there is a minimum 4-partition that does not cross the minimum non-trivial 3-partition.

We use the following idea repeatedly, and hence we make it a proposition.

Proposition 4.1 *Let $\mathcal{X}$ be a minimum partition with property P, and $\mathcal{Y}$ be a minimum partition with property Q. Let $\mathcal{X}'$ and $\mathcal{Y}'$ be a partition with properties P and Q, respectively. If $f(\mathcal{X}) + f(\mathcal{Y}) \geq f(\mathcal{X}') + f(\mathcal{Y}')$, then $\mathcal{Y}'$ is a minimum partition with property Q.*

Proof: Note that $f(\mathcal{X}') \geq f(\mathcal{X})$ and $f(\mathcal{Y}') \geq f(\mathcal{Y})$ because $\mathcal{X}'$ and $\mathcal{Y}'$ have properties P and Q, respectively. Hence we obtain $f(\mathcal{X}') = f(\mathcal{X})$ and $f(\mathcal{Y}) = f(\mathcal{Y}')$. □

In order to simplify the proof and avoid repeating the same set up each time, the following convention is established.

When we try to prove a valid configuration M is good, we always consider a minimum non-trivial 3-partition $\mathcal{X} = \{X_1, X_2, X_3\}$, 3-size minimum 4-partition $\mathcal{Y} = \{Y_1, Y_2, Y_3, Y_3\}$ and a valid submodular function f that generates the configuration M. Let $Z_{ij} = X_i \cap Y_j$ be the *cells* of $\mathcal{X}$ and $\mathcal{Y}$. Let n_i be the number of non-zero values in the ith row of M, and m_j the number of non-zero values in the jth column of M. From this point on, we omit the setup in all the proofs in this section.

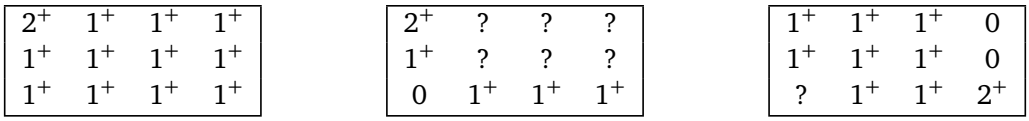
There are simple patterns where the configurations has to be good. The following is a pattern adopted from [14, Lemma 2.5].

Lemma 4.2 *Let M be a configuration. If $M_{11} \geq 2$, and $M_{21}, M_{32}, M_{33}, M_{34} \geq 1$, then M is good.*

Proof: Define $\mathcal{Y}' = \{X_1 \cup X_2 \cup Y_1, Z_{32}, Z_{33}, Z_{34}\}$ and $\mathcal{X}' = \{Z_{11}, Z_{21}, X_3 \cup Y_2 \cup Y_3 \cup Y_4\}$. Then $\mathcal{X}'$ is a non-trivial 3-partition and $\mathcal{Y}'$ is a 4-partition, and $\mathcal{X} \lhd \mathcal{Y}'$. See Figure 4.1 for illustrations.

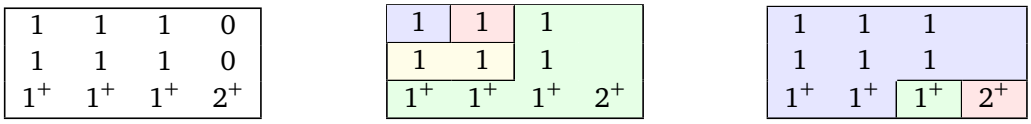
For any submodular function f, it holds that

$$
\begin{aligned}
f(X_1) + f(Y_1) &\geq f(X_1 \cup Y_1) + f(Z_{11}) \\
f(X_1 \cup Y_1) + f(X_2) &\geq f(X_1 \cup X_2 \cup Y_1) + f(Z_{21}) \\
f(X_3) + f(Y_2) &\geq f(X_3 \cup Y_2) + f(Z_{32}) \\
f(X_3 \cup Y_2) + f(Y_3) &\geq f(X_3 \cup Y_2 \cup Y_3) + f(Z_{33}) \\
f(X_3 \cup Y_2 \cup Y_3) + f(Y_4) &\geq f(X_3 \cup Y_2 \cup Y_3 \cup Y_4) + f(Z_{34}).
\end{aligned}
$$



(a) Case 1 (b) Case 2 (c) Case 3

Figure 4.2: The configurations in the proof of Theorem 4.3.



(a) M (b) $\mathcal{Y}'$ (c) $\mathcal{X}'$

Figure 4.3: The configuration M and partitions $\mathcal{Y}'$ and $\mathcal{X}'$ in the Case 3 of the proof of Theorem 4.3.

By summing all the inequalities above, we obtain

$$f(\mathcal{X}) + f(\mathcal{Y}) \geq f(\mathcal{X}') + f(\mathcal{Y}').$$

By Proposition 4.1, $\mathcal{Y}'$ is a min 4-partition, which completes the proof. □

If any configuration equivalent to M satisfies the property in Lemma 4.2, we say M contains a *cross*. If M has many non-zero elements, then M contains a cross. We formalize it below.

Theorem 4.3 *If M is a valid configuration where each row has at least 3 non-zero elements, then it contains a cross, and therefore M is good.*

Proof: We consider the following 3 cases.

Case 1. $m_j = 3$ for all $j \in \{1,2,3,4\}$, then M has at least one entry ≥ 2, and all other values are at least 1. Without loss of generality, assume $M_{11} \geq 2$. See Figure 4.2a. M contains a cross.

Case 2. Assume $m_j \geq 2$ for all j and not all of them are 3. Without loss of generality, assume $M_{31} = 0$. This implies $M_{32}, M_{33}, M_{34} \geq 1$. Since $m_1 = 2$, we have that $M_{11}, M_{21} \geq 1$, and at least one is 2. Without loss of generality, let $M_{11} \geq 2$. See Figure 4.2b. This gives us a cross.

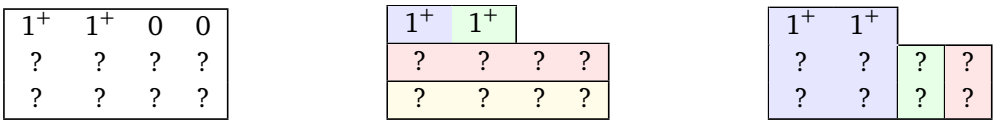
Case 3. Consider $m_j = 1$ for some j. Without loss of generality, we can assume $m_4 = 1$, and $M_{14} = M_{24} = 0$. This shows $M_{34} \geq 2$. We also assume that $M_{32}, M_{33} \geq 1$, because $n_3 \geq 3$.

See Figure 4.2c for the configuration.

If $M_{31} = 0$, then M_{21} or M_{11} is at least 2, and we obtain a cross. Assume next that $M_{31} \geq 1$. if any M_{ij} for $i \in \{1,2\}$ and $j \in \{1,2,3\}$ is 2, then there exists a cross. Hence the only remaining case is when $M_{ij} = 1$ for all $i \in \{1,2\}$ and $j \in \{1,2,3\}$. Let $\mathcal{X}' = \{X_1 \cup X_2 \cup Y_1 \cup Y_2, Z_{33}, Z_{34}\}$ and $\mathcal{Y}' = \{X_3 \cup Y_3 \cup Y_4, Z_{11}, Z_{12}, Z_{21} \cup Z_{22}\}$. $\mathcal{X}'$ is a non-trivial 3-partition. Because $M_{ij} = 1$ for each $i \in \{1,2\}$ and $j \in \{1,2,3\}$, $\mathcal{Y}'$ is a 4-partition. See Figure 4.3 for illustration.

By submodularity, it holds that

$$\begin{aligned}
f(X_3) + f(Y_3) &\geq f(X_3 \cup Y_3) + f(Z_{33}) \\
f(X_3 \cup Y_3) + f(Y_4) &\geq f(X_3 \cup Y_3 \cup Y_4) + f(Z_{34}) \\
f(X_1) + f(Y_1) &\geq f(X_1 \cup Y_1) + f(Z_{11}) \\
f(X_1 \cup Y_1) + f(Y_2) &\geq f(X_1 \cup Y_1 \cup Y_2) + f(Z_{12}) \\
f(X_1 \cup Y_1 \cup Y_2) + f(X_2) &\geq f(X_1 \cup X_2 \cup Y_1 \cup Y_2) + f(Z_{21} \cup Z_{22}).
\end{aligned}$$



(a) M (b) $\mathcal{Y}'$ (c) $\mathcal{X}'$

Figure 4.4: The configuration M and partitions $\mathcal{Y}'$ and $\mathcal{X}'$ in the proof of Lemma 4.4.

According to above, it holds that

$$f(\mathcal{X}) + f(\mathcal{Y}) \geq f(\mathcal{X}') + f(\mathcal{Y}').$$

We invoke Proposition 4.1, and it shows $\mathcal{Y}'$ is a minimum 4-partition. However, $M_{11} = 1$, and hence $\mathcal{Y}'$ is not 3-size. A contradiction to M being a valid configuration. □

The following lemma is an adaptation of [14, Lemma 2.7].

Lemma 4.4 *Let M be a valid configuration with a row with exactly two non-zeros. Then M is good.*

Proof: Without loss of generality, assume $M_{11}, M_{12} \geq 1$ and $M_{13} = M_{14} = 0$. Let $\mathcal{Y}' = \{Z_{11}, Z_{12}, X_2, X_3\}$ and $\mathcal{X}' = \{X_1 \cup Y_1 \cup Y_2, Y_3, Y_4\}$. Note that $\mathcal{Y}'$ is a 4-partition and $\mathcal{X} \lhd \mathcal{Y}'$. Since $\mathcal{Y}$ is 3-size, $|Y_3| \geq 2$, and therefore $\mathcal{X}'$ is a non-trivial 3-partition. See Figure 4.4 for illustration.

By submodularity, it holds that

$$f(X_1) + f(Y_1) \geq f(X_1 \cup Y_1) + f(Z_{11})$$
$$f(X_1 \cup Y_1) + f(Y_2) \geq f(X_1 \cup Y_1 \cup Y_2) + f(Z_{12}).$$

According to above, it holds that

$$f(\mathcal{X}) + f(\mathcal{Y}) \geq f(\mathcal{X}') + f(\mathcal{Y}').$$

By Proposition 4.1, $\mathcal{Y}'$ is a min 4-partition. □

Theorem 4.5 *If M is a valid configuration where there exists a row with at most two non-zero elements, then it is good.*

Proof: If any row has exactly 2 non-zero elements, then we are done by Lemma 4.4.

Hence, consider the case where a row has exactly 1 non-zero element, and no row has exactly 2 non-zero elements. Without loss of generality, let $n_1 = 1$ and $M_{11} \geq 2$.

Case 1: $n_i = 1$ **for some** $i \in \{2, 3\}$. Then M is good because $\mathcal{X} \lhd \mathcal{Y}$.

Case 2: $n_2 = n_3 = 3, m_1 = 1$. Then we have $M_{i1} = 0$ for $i \in \{2, 3\}$ and $M_{i2}, M_{i3}, M_{i4} \geq 1$ for each $i \in \{2, 3\}$. Since $|Y_1| \geq 3$, we have $M_{11} \geq 2$.

Let $\mathcal{Y}' = \{X_2 \cup Y_2, X_1, Z_{33}, Z_{34}\}$ and $\mathcal{X}' = \{X_3 \cup Y_3 \cup Y_4, Y_1, Z_{22}\}$. Then $\mathcal{Y}'$ is a 4-partition, $\mathcal{X} \lhd \mathcal{Y}'$, and $\mathcal{X}'$ is a non-trivial 3-partition. See Figure 4.5 for illustration.

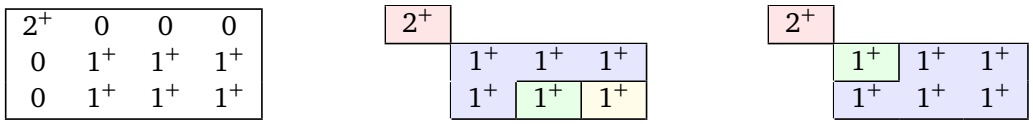
By submodularity, it holds that

$$f(X_2) + f(Y_2) \geq f(X_2 \cup Y_2) + f(Z_{22})$$
$$f(X_3) + f(Y_3) \geq f(X_3 \cup Y_3) + f(Z_{33})$$
$$f(X_3 \cup Y_3) + f(Y_4) \geq f(X_3 \cup Y_3 \cup Y_4) + f(Z_{34}).$$

According to above, it holds that

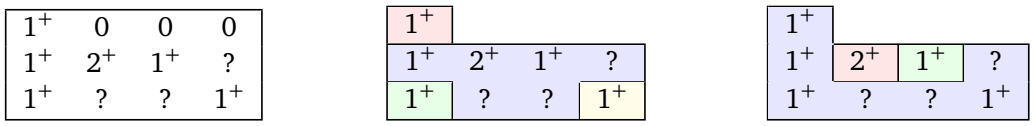
$$f(\mathcal{X}) + f(\mathcal{Y}) \geq f(\mathcal{X}') + f(\mathcal{Y}').$$

By Proposition 4.1, $\mathcal{Y}'$ is a min 4-partition.



(a) M (b) $\mathcal{Y}'$ (c) $\mathcal{X}'$

Figure 4.5: The configuration M and partitions $\mathcal{Y}'$ and $\mathcal{X}'$ in Case 2 of the proof of Theorem 4.5.



(a) M (b) $\mathcal{Y}'$ (c) $\mathcal{X}'$

Figure 4.6: The configuration M and partitions $\mathcal{Y}'$ and $\mathcal{X}'$ in Case 4 of the proof of Theorem 4.5.

Case 3: $n_2, n_3 \geq 3, m_1 = 2$. Without loss of generality, we can assume that $M_{21} \geq 1$ and $M_{31} = 0$. Since $|Y_1| \geq 3$, we have $M_{11} \geq 2$ or $M_{21} \geq 2$, i.e.,

$$M = \begin{array}{|cccc|}\hline 2^+ & 0 & 0 & 0 \\ 1^+ & 1^+ & 1^+ & ? \\ 0 & 1^+ & 1^+ & 1^+ \\ \hline \end{array} \quad \text{or} \quad M = \begin{array}{|cccc|}\hline 1^+ & 0 & 0 & 0 \\ 2^+ & 1^+ & 1^+ & ? \\ 0 & 1^+ & 1^+ & 1^+ \\ \hline \end{array}.$$

In either case, there is a cross, and M is good.

Case 4: $n_1 = 1, n_2, n_3 \geq 3, m_1 = 3$. Without loss of generality, we can assume that $M_{23} \geq 1$. We also assume that $M_{34} \geq 1$ since $n_3 \geq 3$. Since $\mathcal{Y}$ is 3-size and $M_{12} = 0$, without loss of generality, we can assume that $M_{22} \geq 2$. Otherwise, we can exchange Y_3 and Y_4, and then exchange X_2 and X_3. Let $\mathcal{X}' = \{X_3 \cup Y_1 \cup Y_4, Z_{22}, Z_{23}\}$ and $\mathcal{Y}' = \{X_2 \cup Y_2 \cup Y_3, X_1, Z_{31}, Z_{34}\}$. $\mathcal{X}'$ is a non-trivial 3-partition, $\mathcal{Y}'$ is a 4-partition and $\mathcal{X} \lhd \mathcal{Y}'$. See Figure 4.6 for illustration.

By submodularity, it holds that

$$\begin{aligned} f(X_2) + f(Y_2) &\geq f(X_2 \cup Y_2) + f(Z_{22}) \\ f(X_2 \cup Y_2) + f(Y_3) &\geq f(X_2 \cup Y_2 \cup Y_3) + f(Z_{23}) \\ f(X_3) + f(Y_1) &\geq f(X_3 \cup Y_1) + f(Z_{31}) \\ f(X_3 \cup Y_1) + f(Y_4) &\geq f(X_3 \cup Y_1 \cup Y_4) + f(Z_{34}). \end{aligned}$$

According to above, it holds that

$$f(\mathcal{X}) + f(\mathcal{Y}) \geq f(\mathcal{X}') + f(\mathcal{Y}').$$

By Proposition 4.1, $\mathcal{Y}'$ is a minimum 4-partition. □

Theorems 4.3 and 4.5 show that all valid configurations are good, and hence prove Theorem 4.6.

Theorem 4.6 *If all minimum* 4*-partition are* 3*-size, then every minimum non-trivial* 3*-partition is compatible with some minimum* 4*-partition.*

5 Minimum non-trivial 3-partition

In this section, we present an algorithm for computing a minimum nontrivial 3-partition for a given submodular function. This algorithm is based on the following noncrossing property.

2^+	1^+	?
?	?	4^+

Figure 5.1: A configuration M, with a cross that is centered at $(1,3)$. Because $M_{23} \geq 4$, $M_{11}, M_{12} \geq 1$, and $M_{11} \geq 2$.

2^+	1^+	?
?	?	4^+

(a) Case 1

2^+	0	1^+
?	4^+	4^+

(b) Case 2

1	1^-	2^+
4^+	4^+	4^+

(c) Case 3

Figure 5.2: The configurations in the proof of Lemma 5.1.

Lemma 5.1 *Let $f : 2^V \to \mathbb{R}$ be a submodular function with $n \geq 19$, and let $\mathcal{X} = \{X_1, X_2\}$ denote a minimum 4-size 2-partition of f. If f has a minimum nontrivial 3-partition of 5-size, then it contains a minimum nontrivial 3-partition which is noncrossing with $\mathcal{X}$.*

Proof: Let $\mathcal{Y} = \{Y_1, Y_2, Y_3\}$ denote a minimum non-trivial 3-partition of 5-size of f. Let M be the configuration generated by $\mathcal{X}$ and $\mathcal{Y}$. Let $Z_{ij} = X_i \cap Y_j$.

Suppose any row of M has exactly one non-zero. The proof is trivial.

We say that M *contains a cross* if there is i, j such that $M_{i'j} \geq 4$ for $i' \neq i$, $M_{ij'} \geq 1$ for all $j' \neq j$, and $M_{ij'} \geq 2$ for some $j' \neq j$. We also say that *the cross is centered at* (i, j). See Figure 5.1 for an example.

Claim If M contains a cross, then there is a minimum nontrivial 3-partition which is noncrossing with $\mathcal{X}$.

Indeed, without loss of generality, let $M_{11} \geq 2$, $M_{12} \geq 1$ and $M_{23} \geq 4$. Define $\mathcal{Y}' = \{X_2 \cup Y_3, Z_{11}, Z_{12}\}$ and $\mathcal{X}' = \{X_1 \cup Y_1 \cup Y_2, Z_{23}\}$. Certainly $\mathcal{X} \lhd \mathcal{Y}'$. We note that $\mathcal{Y}'$ is a non-trivial 3-partition, and $\mathcal{X}'$ is a 4-size 2-partition. By submodularity,

$$f(X_1) + f(Y_1) \geq f(X_1 \cup Y_1) + f(Z_{11})$$
$$f(X_1 \cup Y_1) + f(Y_2) \geq f(X_1 \cup Y_1 \cup Y_2) + f(Z_{12})$$
$$f(X_2) + f(Y_3) \geq f(X_2 \cup Y_3) + f(Z_{23}).$$

Hence we obtain

$$f(\mathcal{X}) + f(\mathcal{Y}) \geq f(\mathcal{X}') + f(\mathcal{Y}').$$

Therefore, $\mathcal{Y}'$ is a minimum non-trivial 3-partition where $\mathcal{X}$ does not cross it.

Recall the case where each row of M has at least 2 non-zero elements. Because $|V| \geq 19$ and M has 6 entries, at least one entry in M has value at least 4. We can assume $M_{23} \geq 4$. Without loss of generality, let $M_{11} \geq M_{12}$. Next, we show all possible configurations have a cross.

Case 1: $M_{11} \geq 2$, $M_{12} \geq 1$. See Figure 5.2a. There is a cross centered at $(1,3)$.

Case 2: $M_{11} \geq 2$, $M_{12} = 0$. This shows $M_{13} \geq 1$ and $M_{22} \geq 4$. See Figure 5.2b. There is a cross centered at $(1,2)$.

Case 3: $M_{11} = 1$, $M_{12} \leq 1$. This shows $M_{22} \geq 4$. See Figure 5.2c. There is a cross centered at $(1,2)$.

□

In order to use the noncrossing property in Lemma 5.1, we need to compute a minimum 4-size 2-partition of the submodular function f.

Vazirani-Yannakakis [29] proposed an algorithm for enumerating all 2-cuts in a given graph, in the order of nondecreasing weights. Nagamochi-Ibaraki [23] remarked that Vazirani-Yannakakis' algorithm can be extend to enumerating all 2-partitions in an arbitrary system.

For two disjoint sets $S, T \subseteq V$, a family $\{X, V \setminus X\}$ is called an (S,T)-*partition* if $S \subseteq X$ and $T \subseteq V \setminus X$.

Lemma 5.2 (Vazirani-Yannakakis [29], Nagamochi-Ibaraki [23]) *Let $f : 2^V \to \mathbb{R}$ be an arbitrary function with $n = |V|$. All the 2-partitions $\{X, V \setminus X\}$ of V can be enumerated in the order of nondecreasing weights with $O(n\tau^*(n))$-time delay between two consecutive outputs, where $\tau^*(n)$ denotes the time required for computing a minimum (S,T)-partition of f.*

It is well-known that a minimum (S,T)-partition in the submodular function f can be computed in $\tau(n)$ time, i.e., the time required for submodular function minimization. The number of 2-partitions that is not k-size is $O(n^{k-1})$. Thus after enumerating $O(n^{k-1})$ 2-partitions, we can find a minimum k-size 2-partition, which implies the following lemma.

Lemma 5.3 *Let $f : 2^V \to \mathbb{R}$ be a submodular function with $n = |V|$, and let $k (\geq 2)$ denote a positive integer. A minimum k-size 2-partition of f can be computed in $O(n^k \tau(n))$ time.*

We are now ready to describe an algorithm for the minimum nontrivial 3-partition problem. See Figure 5.3. It is similar to both MIN3PARTITION and MIN4PARTITION.

```
MINNONTRIVIAL3PARTITION(f):
  V ← domain(f)
  if |V| ≤ 18
      return the optimum by brute force
  for X ∈ ⋃_{j=1}^{4} (V choose j)
      add candidate {X} ∪ MIN2PARTITION(f_{\X})
  𝒳 ← MIN4SIZE2PARTITION(f)
  for X ∈ 𝒳
      add candidate MINNONTRIVIAL3PARTITION(f_{/X})
  return minimum over all candidates
```

Figure 5.3: Compute a minimum nontrivial-3-partition.

Lemma 5.4 *For a submodular function $f : 2^V \to \mathbb{R}$ with $n = |V|$, there is an algorithm that computes a minimum nontrivial 3 partition of f in $O(n^5 \tau(n))$ time.*

Proof: Consider MINNONTRIVIAL3PARTITION. Finding the minimum 4-size 2-partition takes $O(n^4\tau(n))$ time. Finding the minimum non-5-size 3-partitions takes $O(n^4\tau(n))$ time.

Let $T(n)$ denote the running time of MINNONTRIVIAL3PARTITION. The following recursive relation holds. $T(n) = O(1)$ for $n \leq 18$, and otherwise

$$T(n) = \max_{\substack{a+b=n \\ 1 \leq a \leq b \leq n-4}} T(a+1) + T(b+1) + O(n^4\tau(n)) = O(n^5\tau(n)).$$

□

6 Time complexity of Algorithm MIN4PARTITION

In this section, we analyze the time complexity of Algorithm MIN4PARTITION.

Since the minimum 3-partition problem can be solved in $O(n^3\tau(n))$ time [26], the minimum non-3-size 4-partitions can be found in $O(n^5\tau(n))$ time. A minimum nontrivial 3-partition can be computed in $O(n^5\tau(n))$ time by Lemma 5.4.

Let $T(n)$ be the running time of the 4-partition algorithm in Figure 3.2 with $n = |V|$. The following recursive relation holds. $T(n) = O(1)$ for $n \leq 12$, and otherwise

$$T(n) = \max_{\substack{a+b+c=n \\ 1\leq a\leq b\leq c\leq n-3}} T(a+2) + T(b+2) + T(c+2) + O(n^5\tau(n)) = O(n^6\tau(n)).$$

As a result, we have the following main theorem.

Theorem 6.1 *For a submodular function $f : 2^V \to \mathbb{R}$ with $n = |V|$, Algorithm* MIN4PARTITION(f) *computes a minimum 4-partition of f in $O(n^6\tau(n))$ time.*

As a minimum k-partition in symmetric submodular functions can be found in n^{2k-2} calls to minimum $(k-1)$-partition in submodular functions [4], we observe the following corollary.

Corollary 6.2 *For a symmetric submodular function $f : 2^V \to \mathbb{R}$ with $n = |V|$, a minimum 5-partition can be found in $O(n^{14}\tau(n))$ time.*

7 Concluding remark

In this paper, we have considered the minimum 4-partition problem of submodular functions. Using compatibility for 3- and 4-partitions, we have provided a polynomial-time (exact) algorithm, which settles the complexity status of the problem [11, 22, 23, 26, 31, 34].

It is still open if the minimum k-partition problem is solvable in polynomial time for each fixed k. This is left for future work. Since all our algorithms are very similar, it seems to indicate a generalization to minimum 5-partition problem.

Unfortunately, we remark the obvious generalization of our algorithm does not work for minimum 5-partition. Indeed, there exists a submodular function f with arbitrarily large groundset, such that there exists a minimum 4-partition that is not compatible with any minimum 5-partition. The configuration M generated by them is given as follows:

$$M = \begin{array}{|ccccc|}\hline 1^+ & 0 & 0 & 0 & 0 \\ 1^+ & 0 & 0 & 0 & 0 \\ 1^+ & 1^+ & 1^+ & 0 & 0 \\ 1^+ & 0 & 0 & 1^+ & 1^+ \\ \hline \end{array}.$$

Acknowledgements

This work was partially supported by JSPS KAKENHI Grant Numbers JP19K22841, JP20H00609, and JP20H05967. Chao like to thank Chandra Chekuri and Karthekeyan Chandrasekaran for early discussions.

References

[1] Calvin Beideman, Karthekeyan Chandrasekaran, and Weihang Wang. Counting and Enumerating Optimum Cut Sets for Hypergraph k-Partitioning Problems for Fixed k. In Mikołaj Bojańczyk, Emanuela Merelli, and David P. Woodruff, editors, *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, volume 229 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 16:1–16:18, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[2] Calvin Beideman, Karthekeyan Chandrasekaran, and Weihang Wang. Deterministic enumeration of all minimum k-cut-sets in hypergraphs for fixed k. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2208–2228, Philadelphia, PA, January 2022. Society for Industrial and Applied Mathematics.

[3] Calvin Beideman, Karthekeyan Chandrasekaran, and Chao Xu. Multicriteria cuts and size-constrained k-cuts in hypergraphs. *Mathematical Programming*, November 2021.

[4] Karthekeyan Chandrasekaran and Chandra Chekuri. Min-max partitioning of hypergraphs and symmetric submodular functions. In *Proceedings of the Thirty-Second Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '21, page 1026–1038, USA, 2021. Society for Industrial and Applied Mathematics.

[5] Karthekeyan Chandrasekaran and Chandra Chekuri. Hypergraph k-Cut for Fixed k in Deterministic Polynomial Time. *Mathematics of Operations Research*, page moor.2021.1250, March 2022.

[6] Karthekeyan Chandrasekaran, Chao Xu, and Xilin Yu. Hypergraph k-cut in randomized polynomial time. *Mathematical Programming*, 186(1-2):85–113, March 2021.

[7] Chandra Chekuri and Alina Ene. Approximation Algorithms for Submodular Multiway Partition. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*, pages 807–816, Palm Springs, CA, USA, October 2011. IEEE.

[8] Chandra Chekuri, Kent Quanrud, and Chao Xu. LP Relaxation and Tree Packing for Minimum k-Cut. *SIAM Journal on Discrete Mathematics*, 34(2):1334–1353, January 2020.

[9] Chandra Chekuri and Chao Xu. Minimum Cuts and Sparsification in Hypergraphs. *SIAM Journal on Computing*, 47(6):2118–2156, January 2018.

[10] Kyle Fox, Debmalya Panigrahi, and Fred Zhang. Minimum cut and minimum k-cut in hypergraphs via branching contractions. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 881–896. SIAM, 2019.

[11] Takuro Fukunaga. Computing minimum multiway cuts in hypergraphs. *Discrete Optimization*, 10(4):371–382, November 2013.

[12] Leszek Gasieniec, Jesper Jansson, Andrzej Lingas, and Anna Östlin. On the Complexity of Constructing Evolutionary Trees. *Journal of Combinatorial Optimization*, 3(2/3):183–197, 1999.

[13] O. Goldschmidt and D. S. Hochbaum. A Polynomial Algorithm for the k-cut Problem for Fixed k. *Mathematics of Operations Research*, 19:24–37, 1994.

[14] F. Guinez and M. Queyranne. The size-constrained submodular k-partition problem. Unpublished manuscript. Available at https://docs.google.com/viewer?a=v&pid=sites&srcid=ZGVmYXVsdGRvbWFpbnxmbGF2aW9ndWluZXpob21lcGFnZXxneDo0NDVlMThkMDg4ZWRlOGI1. See also https://smartech.gatech.edu/bitstream/handle/1853/43309/Queyranne.pdf, 2012.

[15] Anupam Gupta, David G. Harris, Euiwoong Lee, and Jason Li. Optimal bounds for the k-cut problem. *J. ACM*, 69(1), nov 2021.

[16] Anupam Gupta, Euiwoong Lee, and Jason Li. Faster exact and approximate algorithms for k-cut. In *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 113–123, 2018.

[17] Yoko Kamidoi, Noriyoshi Yoshida, and Hiroshi Nagamochi. A deterministic algorithm for finding all minimum k-way cuts. *SIAM J. Comput.*, 36(5):1329–1341, 2006.

[18] Regina Klimmek and Frank Wagner. A simple hypergraph min cut algorithm. Technical Report B 96-02, FU Berlin Fachbereich Mathematik und Informatik, 1996.

[19] Yin Tat Lee, Aaron Sidford, and Sam Chiu-Wai Wong. A faster cutting plane method and its implications for combinatorial and convex optimization. In *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*, pages 1049–1065, 2015.

[20] Wai K. Mak and D. F. Wong. Fast hypergraph min-cut algorithm for circuit partitioning. *Integration, the VLSI Journal*, 30(1):1–11, 2000.

[21] H Nagamochi and T Ibaraki. Computing edge-connectivity in multigraphs and capacitated graphs. *SIAM Journal on Discrete Mathematics*, 5(1):54–66, 1992.

[22] Hiroshi Nagamochi. Algorithms for the minimum partitioning problems in graphs. *Electronics and Communications in Japan (Part III Fundamental Electronic Science)*, 90(10):63–78, 2007.

[23] Hiroshi Nagamochi and Toshihide Ibaraki. A fast algorithm for computing minimum 3-way and 4-way cuts. *Mathematical Programming*, 88(3):507–520, 2000.

[24] Hiroshi Nagamochi, Shigeki Katayama, and Toshihide Ibaraki. A Faster Algorithm for Computing Minimum 5-Way and 6-Way Cuts in Graphs. *Journal of Combinatorial Optimization*, 4(2):151–169, 2000.

[25] Kazumasa Okumoto, Takuro Fukunaga, and Hiroshi Nagamochi. Divide-and-conquer algorithms for partitioning hypergraphs and submodular systems. In Yingfei Dong, Ding-Zhu Du, and Oscar Ibarra, editors, *Algorithms and Computation*, pages 55–64, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.

[26] Kazumasa Okumoto, Takuro Fukunaga, and Hiroshi Nagamochi. Divide-and-Conquer Algorithms for Partitioning Hypergraphs and Submodular Systems. *Algorithmica (New York)*, pages 1–20, 2010.

[27] Maurice Queyranne. Minimizing symmetric submodular functions. *Mathematical Programming*, 82(1):3–12, 1998.

[28] Mikkel Thorup. Minimum k-way cuts via deterministic greedy tree packing. *Proceedings of the 40th annual ACM Symposium on Theory of Computing*, pages 159–165, 2008.

[29] Vijay V Vazirani and Mihalis Yannakakis. Suboptimal cuts: Their enumeration, weight and number. In *International Colloquium on Automata, Languages, and Programming*, pages 366–377. Springer, 1992.

[30] Mingyu Xiao. An improved divide-and-conquer algorithm for finding all minimum k-way cuts. In Seok-Hee Hong, Hiroshi Nagamochi, and Takuro Fukunaga, editors, *Algorithms and Computation*, pages 208–219, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.

[31] Mingyu Xiao. Finding minimum 3-way cuts in hypergraphs. *Information Processing Letters*, 110(14-15):554–558, 2010.

[32] Li Pu Yeh, Biing Feng Wang, and Hsin Hao Su. Efficient algorithms for the problems of enumerating cuts by non-decreasing weights. *Algorithmica (New York)*, 56(3):297–312, 2010.

[33] Liang Zhao, Hiroshi Nagamochi, and Toshihide Ibaraki. On generalized greedy splitting algorithms for multiway partition problems. *Discrete Applied Mathematics*, 143(1):130–143, 2004.

[34] Liang Zhao, Hiroshi Nagamochi, and Toshihide Ibaraki. Greedy splitting algorithms for approximating multiway partition problems. *Mathematical Programming*, 102(1):167–183, January 2005.